

2. Block World Problem

```
from collections import deque

# Function to check whether a block is clear
def is_clear(state, block):
    for b, position in state:
        if position == block:
            return False
    return True

# Generate possible moves
def generate_moves(state, blocks):
    moves = []

    for block in blocks:

        # Block must be clear
        if not is_clear(state, block):
            continue

        # Find current position
        current_position = None

        for b, position in state:
```

```
if b == block:
    current_position = position
    break
```

```
# Move block to Table
```

```
if current_position != "Table":
```

```
    new_state = set(state)
    new_state.remove((block, current_position))
    new_state.add((block, "Table"))
```

```
    moves.append(
        (frozenset(new_state),
         f"Move {block} from {current_position} to Table")
    )
```

```
# Move block onto another block
```

```
for destination in blocks:
```

```
    if block == destination:
        continue
```

```
# Destination must be clear
```

```
if not is_clear(state, destination):
    continue
```

```

        # Don't move to the same position
        if current_position == destination:
            continue

        new_state = set(state)

        new_state.remove((block, current_position))
        new_state.add((block, destination))

        moves.append(
            (frozenset(new_state),
             f"Move {block} from {current_position} to {destination}")
        )

    return moves

# BFS function
def block_world(initial_state, goal_state, blocks):

    queue = deque()
    queue.append((frozenset(initial_state), []))

    visited = set()
    visited.add(frozenset(initial_state))

    while queue:

```

```
current_state, path = queue.popleft()
```

```
# Check goal
```

```
if current_state == frozenset(goal_state):
```

```
    return path
```

```
# Generate next states
```

```
for new_state, move in generate_moves(current_state, blocks):
```

```
    if new_state not in visited:
```

```
        visited.add(new_state)
```

```
        new_path = path + [move]
```

```
        queue.append((new_state, new_path))
```

```
return None
```

```
# ----- MAIN PROGRAM -----
```

```
print("==== BLOCK WORLD PROBLEM ====")
```

```
# Number of blocks
```

```
n = int(input("Enter number of blocks: "))
```

```
blocks = []
```

```
print("\nEnter block names:")
```

```
for i in range(n):
```

```
    block = input(f"Block {i + 1}: ")
```

```
    blocks.append(block)
```

```
# Initial state
```

```
print("\n===== INITIAL STATE =====")
```

```
print("Enter the position of each block.")
```

```
print("Enter 'Table' if the block is on the table.")
```

```
initial_state = set()
```

```
for block in blocks:
```

```
    position = input(f"Where is {block} initially? ")
```

```
    initial_state.add((block, position))
```

```
# Goal state
```

```
print("\n===== GOAL STATE =====")
```

```
print("Enter the desired position of each block.")
```

```
goal_state = set()
```

```
for block in blocks:
```

```
position = input(f"Where should {block} be in the goal state? ")
goal_state.add((block, position))

# Display states
print("\nInitial State:")
for block, position in initial_state:
    print(block, "->", position)

print("\nGoal State:")
for block, position in goal_state:
    print(block, "->", position)

# Solve
solution = block_world(initial_state, goal_state, blocks)

# Display solution
if solution:

    print("\n==== SOLUTION =====")

    for i, move in enumerate(solution, 1):
        print(f"Step {i}: {move}")

    print("\nTotal moves:", len(solution))

else:
```

```
print("\nNo solution found.")
```

3. Depth Limited Search (DLS) and Uniform Cost Search (UCS)

Depth Limited Search (DLS):

Depth Limited Search

```
def dls(graph, current, goal, limit, path):
```

```
    # Add current node to path
```

```
    path.append(current)
```

```
    # Goal found
```

```
    if current == goal:
```

```
        return True
```

```
    # Depth limit reached
```

```
    if limit == 0:
```

```
        path.pop()
```

```
        return False
```

```
    # Visit neighboring nodes
```

```
    for neighbor in graph[current]:
```

```
        if neighbor not in path:
```

```
    if dls(graph, neighbor, goal, limit - 1, path):  
        return True
```

```
# Backtrack
```

```
path.pop()
```

```
return False
```

```
# Main Program
```

```
print("==== DEPTH LIMITED SEARCH ====")
```

```
n = int(input("Enter number of nodes: "))
```

```
graph = {}
```

```
# Create nodes
```

```
for i in range(n):
```

```
    graph[i] = []
```

```
# Enter edges
```

```
e = int(input("Enter number of edges: "))
```

```
print("Enter edges (source destination):")
```

```
for i in range(e):
```



```
u, v = map(int, input().split())
```

```
graph[u].append(v)
```

```
graph[v].append(u)
```

```
start = int(input("Enter starting node: "))
```

```
goal = int(input("Enter goal node: "))
```

```
limit = int(input("Enter depth limit: "))
```

```
path = []
```

```
# Perform DLS
```

```
found = dls(graph, start, goal, limit, path)
```

```
# Display result
```

```
if found:
```

```
    print("\nGoal Found!")
```

```
    print("Path:", " -> ".join(map(str, path)))
```

```
    print("Depth:", len(path) - 1)
```

```
else:
```

```
    print("\nGoal not found within the given depth limit.")
```

Uniform Cost Search (UCS):

```
import heapq

# Uniform Cost Search
def ucs(graph, start, goal):

    # Priority queue
    priority_queue = []

    # (cost, node, path)
    heapq.heappush(priority_queue, (0, start, [start]))

    # Store the lowest cost found for each node
    visited = {}

    while priority_queue:

        cost, current, path = heapq.heappop(priority_queue)

        # If node was already reached with lower cost
        if current in visited and visited[current] <= cost:
            continue
```

```
visited[current] = cost
```

```
# Goal found
```

```
if current == goal:
```

```
    return path, cost
```

```
# Explore neighbors
```

```
for neighbor, edge_cost in graph[current]:
```

```
    new_cost = cost + edge_cost
```

```
    new_path = path + [neighbor]
```

```
    heapq.heappush(
        priority_queue,
        (new_cost, neighbor, new_path)
    )
```

```
return None, float('inf')
```

```
# Main Program
```

```
print("==== UNIFORM COST SEARCH =====")
```

```
n = int(input("Enter number of nodes: "))
```

```
graph = {}
```

```
for i in range(n):
```

```
    graph[i] = []
```

```
# Enter edges
```

```
e = int(input("Enter number of edges: "))
```

```
print("Enter edges as:")
```

```
print("source destination cost")
```

```
for i in range(e):
```

```
    u, v, cost = map(int, input().split())
```

```
    graph[u].append((v, cost))
```

```
    graph[v].append((u, cost))
```

```
start = int(input("Enter starting node: "))
```

```
goal = int(input("Enter goal node: "))
```

```
# Perform UCS
```

```
path, cost = ucs(graph, start, goal)
```

```
# Display result
```

```
if path:
```

```
    print("\nGoal Found!")
```

```
    print("Optimal Path:",  
          " -> ".join(map(str, path)))
```

```
    print("Minimum Cost:", cost)
```

```
else:
```

```
    print("\nGoal not found.")
```